

Supply-Aware Promotion Uplift

Mã nguồn tham khảo cho simulator: FleetPy và các simulator mở

Nhóm thực hiện: Nguyễn Huy Hoàng, Phan Văn Tình

Người hướng dẫn: Nguyễn Bảo Long

Tóm tắt

Đề tài có 5 tuần và đã có sẵn `marketplace_sim.py`: simulator này đã đáp ứng RQ1–RQ3 (có confounding, interference, và tính $V(\pi)$ trong khoảng 7 giây cho 30 ngày). Vì vậy **simulator chính của đề tài là `marketplace_sim.py`**. Thiết kế tác tử 13 module (M1–M13) chỉ được làm, nếu còn thời gian, dưới dạng **bản rút gọn để kiểm tra ngoài** (external check), không thay simulator chính. Báo cáo này rà soát khoảng 80 kho mã, đọc hơn 20 kho ở mức mã nguồn và chạy thử khoảng 15 kho để trả lời: phần nào của bản rút gọn có thể dựa vào mã mở. Kết luận: **không có mã mở nào thay được simulator của nhóm**, vì không kho nào có cùng lúc voucher, hủy chuyển theo ETA, lưới H3 và thiết kế thí nghiệm. **FleetPy** (Engelhardt et al., 2022) là **ứng viên tham chiếu kiến trúc** (MIT, còn bảo trì, đủ vòng vận hành đội xe), nhưng chỉ được cam kết sau một **spike ≤ 1 ngày**. Các thành phần ngoài phạm vi đề bài (surge, repositioning theo cầu, học tăng cường) được loại khỏi kế hoạch. Các lần chạy thử trên **mô hình test** (chưa chạy trên simulator thật của nhóm) gợi ý rằng chọn sai đơn vị ngẫu nhiên hoá có thể làm ước lượng lệch nhiều lần, thậm chí đổi dấu.

Mục lục

1	Giới thiệu	2
2	Mã nguồn trong các nghiên cứu liên quan	2
3	Ứng viên tham chiếu kiến trúc: FleetPy	3
4	Khám phá mã nguồn	4
4.1	Toàn cảnh các simulator mở	4
4.2	Kiến trúc FleetPy	5
4.3	Các thành phần đáng lấy trong FleetPy	5
4.4	Ma trận năng lực của các nguồn mã	6
4.5	Kết quả chạy thử trên mô hình test	6
5	Áp dụng mã nguồn cho bản ABM rút gọn	9
5.1	Nguồn mã cho từng module	9
5.2	Thư viện dùng trực tiếp	10
6	Kết luận	11

1 Giới thiệu

Báo cáo thiết kế đã mô tả một simulator tác tử gồm 6 khối và 13 module (M1–M13) (Nhóm đề tài (Simulator), 2026). Với thời lượng 5 tuần, nhóm không dựng lại toàn bộ thiết kế này làm simulator chính:

Vai trò của các simulator trong đề tài

- **Simulator chính: `marketplace_sim.py`.** Đã có confounding từ quy tắc phát voucher cũ, interference qua ràng buộc cung, và hàm $V(\pi)$ chạy khoảng 7 giây cho 30 ngày; đủ để trả lời RQ1–RQ3. Mọi kết quả chính của đề tài lấy từ simulator này.
- **Bản ABM rút gọn (tuỳ chọn).** Chỉ làm khi phần chính đã xong, với số module tối thiểu, để kiểm tra ngoài xem các kết luận có còn đúng dưới một cơ chế ghép chuyển chi tiết hơn hay không. Báo cáo này phục vụ bản rút gọn đó.

Mục tiêu của báo cáo là trả lời: *nếu làm bản rút gọn, module nào có thể dựa vào mã nguồn mở, dựa vào đâu, và được phép dùng tới mức nào*. Phạm vi được giới hạn theo đề bài (§6); Bảng 1 liệt kê các module bị ảnh hưởng. Mỗi nguồn mã được đánh giá theo bốn mức tái sử dụng (Bảng 2).

Bảng 1. Các thành phần nằm ngoài phạm vi đề bài (§6) và cách xử lý trong simulator.

Thành phần	Module	Cách xử lý
Surge pricing	M3	Bỏ. Giá phản ứng với cung/cầu sẽ tương tác với voucher và làm khó tách hiệu ứng voucher. Giá chỉ theo thời gian/quãng đường.
Repositioning theo cầu	M9	Không làm cung nội sinh với voucher. Nếu giữ M9, dùng quy tắc cố định (không phụ thuộc voucher).
Incentive phía tài xế	M8	Cung ngoại sinh: số xe theo ca cho trước, không có quyết định vào/ra theo thu nhập.
Học tăng cường, quyết định tuần tự	–	Không đầu tư thêm. Kết quả PPO ở Mục 4.5 chỉ ghi lại để tham khảo.

Bảng 2. Bốn mức tái sử dụng mã nguồn.

Mức	Ý nghĩa	Điều kiện
Dùng	Thêm làm thư viện phụ thuộc của dự án	Giấy phép dễ dãi, còn bảo trì
Copy	Sao chép mã, giữ thông báo bản quyền	MIT, BSD, Apache
Port	Chuyển công thức sang Python, ghi nguồn	Có giấy phép (kể cả CC-BY)
Đọc	Chỉ đọc cơ chế, tự viết lại từ bài báo	Kho không có giấy phép

2 Mã nguồn trong các nghiên cứu liên quan

Phần lớn nghiên cứu về thí nghiệm trên thị trường gọi xe dựng simulator riêng; chỉ một số công bố mã (Bảng 3).

Bảng 3. Mã nguồn đi kèm các nghiên cứu và simulator mở tiêu biểu.

Nguồn	Mã, giấy phép	Dùng để
Engelhardt et al. (2022)	FleetPy; Python, MIT	Mô phỏng và điều khiển đội xe theo yêu cầu: dispatch, pooling, repositioning, sạc, giá
Farias et al. (2022)	rideshare-simulator (kèm bài)	Simulator Manhattan đo độ chệch thí nghiệm khi có interference
Peng et al. (2025)	or-gymnax, xp_gym; JAX, không có file LICENSE	Simulator định giá Manhattan; so sánh bảy bộ ước lượng switchback
Castillo (2025)	Gói tái lập; R/C++, CC-BY-4.0	Thị trường thành phố trên lưới H3: logit theo giá và ETA, surge, wild goose chase
Shi et al. (2023)	CausalMAREL; Python, không có giấy phép	Trợ giá theo vùng, tài xế di chuyển sang vùng kề; đánh giá off-policy
Chen et al. (2025)	HRSim; Python, MIT	Nền tảng tác tử cho ride-sharing sức chứa cao
tomslee/ridehail	Python, MIT	Mô hình nhỏ tái hiện wild goose chase; giá một mức

Các simulator công nghiệp (Uber, Lyft, DoorDash, DiDi, Grab) chỉ được mô tả qua blog kỹ thuật, không công bố mã. Simulator KDD Cup 2020 của DiDi cũng chưa từng được công bố.

3 Ứng viên tham chiếu kiến trúc: FleetPy

Đề xuất: FleetPy là ứng viên tham chiếu kiến trúc cho bản ABM rút gọn vì

- (1) **Đủ vòng vận hành đội xe** trong một khung Python: sinh yêu cầu, báo giá, ghép chuyến, đón-chở-trả, hủy, repositioning, định giá ([Engelhardt et al., 2022](#)); đúng các khối B, C, D trong thiết kế của nhóm.
- (2) **Còn bảo trì và giấy phép MIT**: bản 1.0.2 (03/2026), vẫn được cập nhật trong 2026; được sao chép và sửa mã.
- (3) **Tách engine khỏi chính sách**: phần xe thực thi và phần operator lập kế hoạch là hai lớp riêng, nên thay chính sách voucher mà không đụng vòng mô phỏng.
- (4) **Đã được thiết kế của nhóm tham khảo**: vòng lặp theo tick, bảng thời gian di chuyển và cách ghi log đều dẫn FleetPy ([Nhóm đề tài \(Simulator\), 2026](#)).
- (5) **Có sẵn dữ liệu Manhattan và Chicago (CC-BY-4.0)** để kiểm tra chéo với dữ liệu NYC TLC mà nhóm dùng.

FleetPy **không** được fork làm lỗi: nó không có voucher, gán can thiệp có confounding, switchback hay chạy phần thực, và batch ILP cần solver (Gurobi hoặc OR-Tools). Nhóm lấy kiến trúc và công thức, không lấy nguyên khung.

Chưa cài thử: cần spike trước khi cam kết

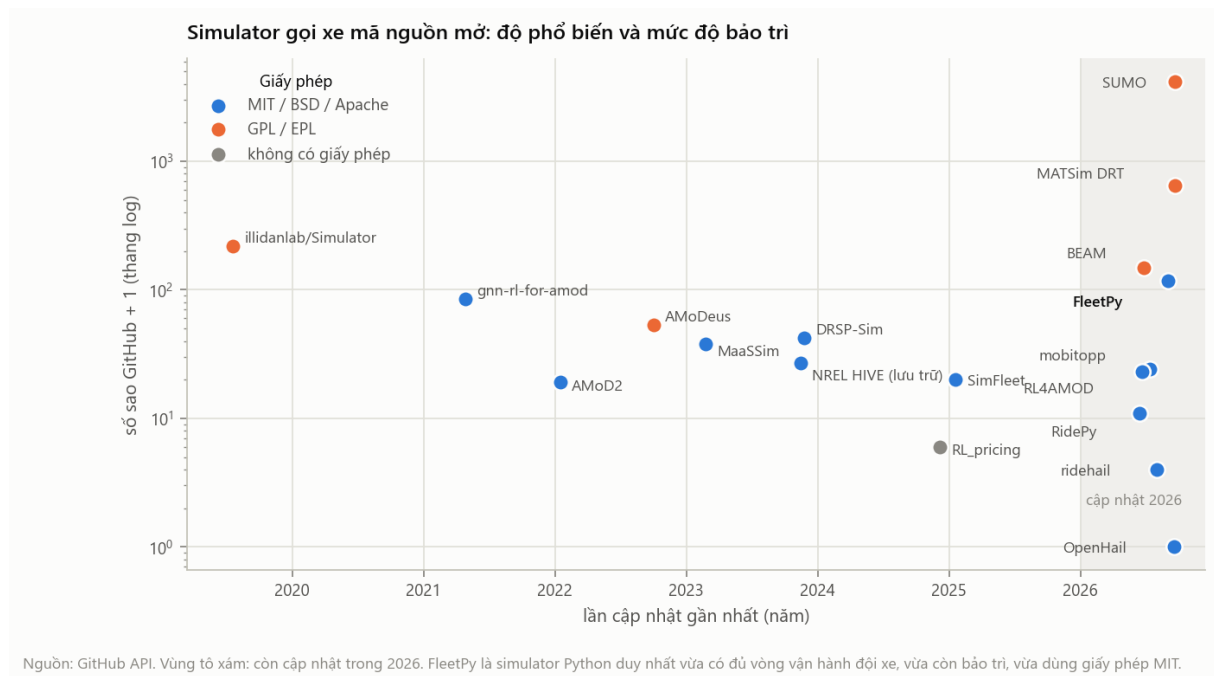
Nhóm **chưa cài thử FleetPy**, nên chọn nó làm tham chiếu kiến trúc lúc này là rủi ro. Trước khi cam kết, nhóm làm một **spike tối đa 1 ngày**: cài từ GitHub, chạy ví dụ Manhattan, đo thời gian chạy và ghi lại các trở ngại (solver, phiên bản thư viện). Tiêu chí dừng: nếu sau 1 ngày chưa chạy được ví dụ, hoặc thời gian chạy một ngày mô phỏng quá lâu so với nhu cầu chạy nhiều seed, nhóm bỏ FleetPy và chỉ đọc công thức từ bài báo ([Engelhardt et al., 2022](#)).

Bảng 4. Thông tin cơ bản về FleetPy.

Kho mã	https://github.com/TUM-VT/FleetPy (TUM-VT, Đại học Kỹ thuật München)
Ngôn ngữ, giấy phép	Python, bộ định tuyến C++ tùy chọn; MIT
Mức độ hoạt động	116 sao, 48 fork; phát hành 1.0.1 (08/2025) và 1.0.2 (03/2026); vẫn được cập nhật trong 2026
Tài liệu	README, wiki “FleetPy for dummies”, các nghiên cứu mẫu, module test trong <code>studies/module_tests/</code>
Quy mô đã công bố	Manhattan, 380 xe sức chứa 4, mẫu 10% chuyển taxi NYC, tối ưu lại mỗi 60 giây
Lưu ý cài đặt	Gói <code>fleetpy</code> trên PyPI là gói khác (2015); phải cài từ GitHub

4 Khám phá mã nguồn

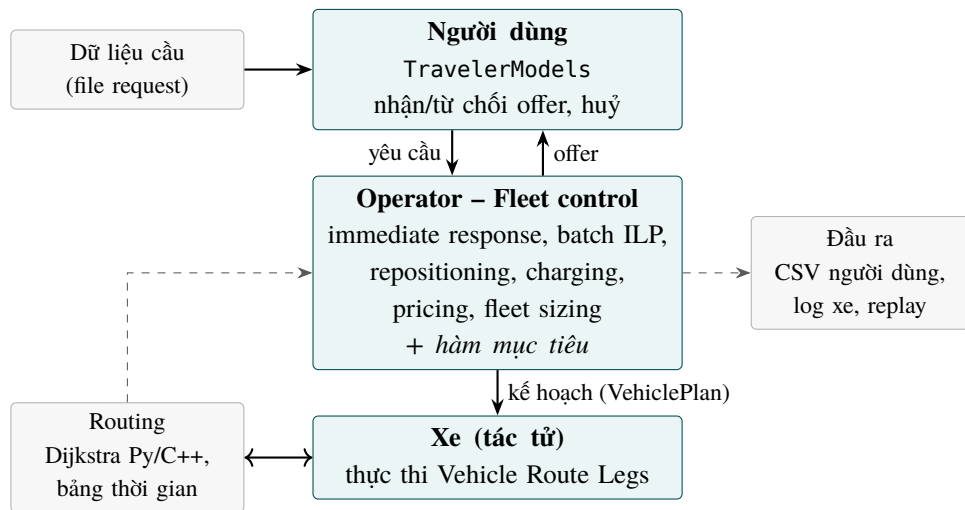
4.1 Toàn cảnh các simulator mở

**Hình 1.** Độ phổ biến và mức độ bảo trì của các simulator gọi xe mã nguồn mở.

- Các nền tảng lớn còn bảo trì (SUMO, MATSim, BEAM) chạy trên C++/Java/Scala và dùng GPL/EPL: mạnh nhưng nặng, khó nhúng vào dự án Python.
- Phần lớn simulator Python đi kèm một bài báo đã ngừng cập nhật từ 2020–2023 (MaaSSim, AMoD2, DRSP-Sim; NREL HIVE bị lưu trữ từ 06/2026).
- **FleetPy** nằm ở vùng tốt nhất: Python, MIT, còn cập nhật trong 2026, đủ vòng vận hành.

► **Dùng cho simulator:** FleetPy là ứng viên tham chiếu kiến trúc cho bản rút gọn (sau spike); các kho còn lại chỉ lấy từng công thức.

4.2 Kiến trúc FleetPy



Hình 2. Kiến trúc hai lớp của FleetPy: xe thực thi, operator lập kế hoạch và chấm điểm bằng hàm mục tiêu.

- **Xe** là tác tử thực thi các *Vehicle Route Legs* (lái, đón/trả khách, sạc). **Operator** lập kế hoạch trên đối tượng giả định *PlanRequest/VehiclePlan* và chấm điểm bằng một hàm mục tiêu tùy biến.
- Hai chế độ tương tác: yêu cầu → operator quyết định; hoặc yêu cầu → operator báo giá → người dùng quyết định. Chế độ thứ hai khớp với trình tự “báo giá + voucher → rider quyết định đặt” của nhóm.

► **Dùng cho simulator:** mẫu tách khối C (chính sách nền tảng) khỏi khối D (vận hành đội xe); quy ước session báo giá rồi mới tạo order.

4.3 Các thành phần đáng lấy trong FleetPy

Bảng 5. Thành phần FleetPy và module tương ứng trong simulator của nhóm.

Thành phần FleetPy	Nội dung	Module nhóm
Bảng thời gian di chuyển, bộ nhớ đệm OD	Dijkstra Python/C++; ma trận tính trước toàn phần hoặc một phần	M1 SpaceTime
<code>TravelerModels.py</code>	Các lớp hành vi: nhảy giá, giảm dần theo thời gian chờ, chọn theo utility $U = -\text{vot } t_{\text{wait}} - \text{vot } t_{\text{drive}} - \text{fare}$	M4 RiderChoice
<code>_compute_fare()</code>	Giá = (cước mở cửa, quãng đường, thời gian) × hệ số mức sử dụng × hệ số surge theo zone (nhóm bỏ hệ số surge)	M3 Pricing
Immediate response, batch	Insertion heuristic; biến thể Alonso-Mora và ILP cho ghép lô	M5 Matching
Vehicle Route Legs	Lịch trình đón–chờ–trả, giữ xe khi bận	M6 TripExecution
<code>cancels_booking()</code>	Hook hủy sau khi đã nhận offer	M7 Cancellation
Fleet sizing, repositioning	Ca tài xế; điều chuyển theo dự báo cầu (Zhang & Pavone; Syed et al.)	M8, M9
Đầu ra CSV, <code>replay_pyplot.py</code>	Thống kê người dùng, log nhiệm vụ xe, công cụ replay	M12 Logger

Hạn chế cần biết: quy mô công bố chỉ 380 xe; thời gian di chuyển tất định, không phản hồi tắc nghẽn do chính đội xe; mô hình sạc đơn giản. Các thông tin trong mục này lấy từ tài liệu và mã nguồn, chưa được

kiểm chứng bằng cài đặt (xem spike ở Mục 3).

4.4 Ma trận năng lực của các nguồn mã

Năng lực của từng nguồn mã so với thiết kế simulator của nhóm										
	Castillo	RL_pricing	aaivu	MaaSSim fork	xp_gym	Causal MARL	ridehail	OpenHail	FleetPy	Thiết kế nhóm
Lưới hex / H3	✓	–	~	–	–	~	–	–	–	✓
Khung giờ trong ngày	✓	✓	✓	~	✓	✓	~	✓	✓	✓
P(đất) theo giá và ETA	✓	~	–	✓	✓	–	~	–	✓	✓
Huỷ / bỏ chuyến	~	✓	✓	~	–	–	~	~	✓	✓
Surge / giá động	✓	✓	~	–	~	–	~	–	✓	~
Voucher / trợ giá	–	–	–	✓	–	~	–	–	~	✓
Incentive tài xế	~	~	✓	✓	–	~	~	–	~	~
Matching	✓	✓	✓	✓	✓	~	✓	✓	✓	✓
Wild goose chase	✓	–	–	–	–	–	✓	–	~	✓
Spillover giữa vùng	✓	✓	~	~	✓	✓	–	~	✓	✓
API MDP / Gym	–	~	~	–	✓	~	–	✓	–	–
Switchback	–	–	–	–	✓	~	–	–	–	✓

Ở xanh đậm: có sẵn · xanh nhạt: một phần · xám: không có. Cột cuối lấy theo báo cáo thiết kế simulator (M1–M13). Nguồn: đọc mã và chạy thử. Không nguồn mở nào phủ đủ; thiết kế của nhóm ghép các mảnh lại.

Hình 3. Năng lực của từng nguồn mã so với thiết kế simulator của nhóm.

- Không nguồn nào phủ đủ 12 năng lực. [Castillo \(2025\)](#) phủ nhiều nhất (H3, giá và ETA, surge, wild goose chase) nhưng không có voucher; chỉ nhánh MaaSSim có voucher; chỉ xp_gym có switchback kèm API dạng Gym.
- FleetPy mạnh ở vận hành (huỷ, matching, giá, spillover) nhưng thiếu lưới hex, voucher và thiết kế thí nghiệm.

► **Dùng cho simulator:** mỗi năng lực còn thiếu được ghép từ nguồn phủ tốt nhất (Mục 5).

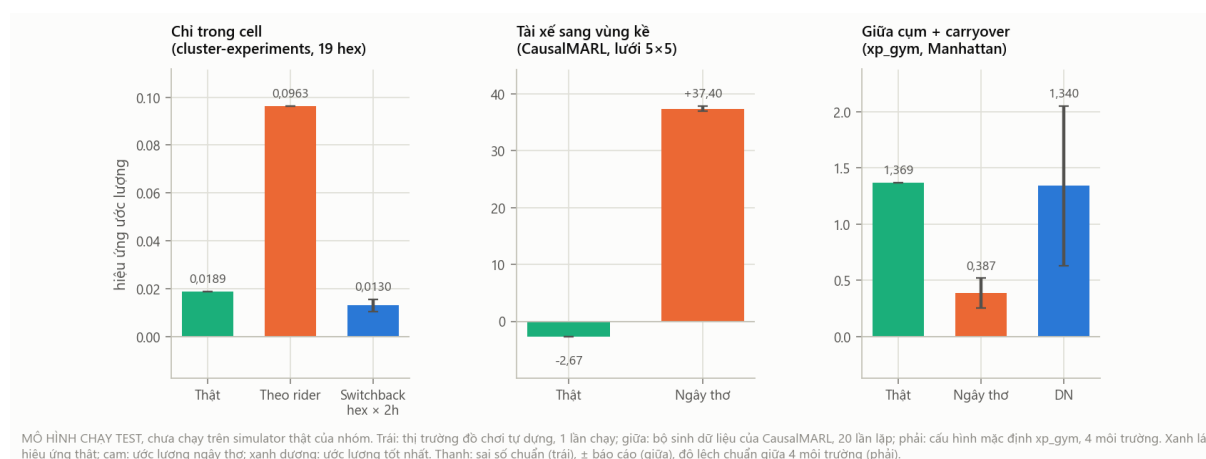
4.5 Kết quả chạy thử trên mô hình test

Lưu ý: mô hình chạy test, chưa chạy thật

Các kết quả trong mục này là **chạy thử trên mô hình test** để kiểm tra mã nguồn chạy được và minh hoạ cơ chế: thị trường đồ chơi tự dựng, bộ sinh dữ liệu mẫu của tác giả, hoặc cấu hình mặc định của từng kho, với số lần chạy ít (1 đến 20 lần, xem Bảng 6). Chúng **chưa phải kết quả chạy trên simulator thật của nhóm** và các tham số chưa được hiệu chỉnh, nên chỉ dùng để định hướng thiết kế, không dùng làm kết luận định lượng.

Bảng 6. Số lần chạy và độ bất định của từng kết quả chạy thử. KTC 95%: khoảng tin cậy 95% xấp xỉ (trung bình $\pm 1,96$ SE).

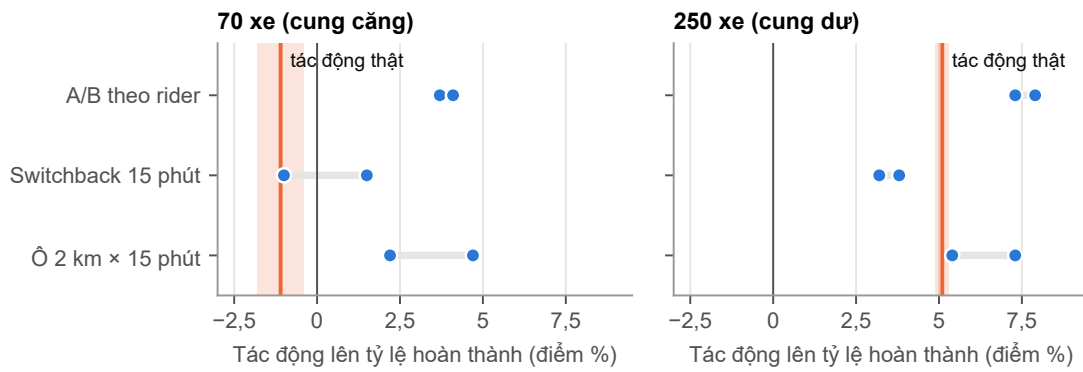
Kết quả	Số lần chạy	Giá trị và độ bất định
Chỉ trong cell (Hình 4, trái)	1 lần chạy (1 seed)	Thật 0,0189; theo rider 0,0963; switchback 0,0130 (SE 0,0025; KTC 95% 0,008–0,018)
Tài xế sang vùng kề (giữa)	20 lần Monte Carlo	Thật $-2,67$; ngẫu thơ $+37,4 \pm 0,45$
Có carryover (phải)	4 môi trường	Thật 1,369; ngẫu thơ 0,387; DN $1,340 \pm 0,710$ (độ lệch chuẩn giữa 4 môi trường)
Simulator Farias (Hình 5)	2 seed mỗi cấu hình	Chưa đủ để tính khoảng tin cậy
Qini/AUUC (Hình 6)	1 bộ dữ liệu tổng hợp, hoán vị thứ tự dòng	Qini/ n dao động 0,00319–0,00367 theo thứ tự dòng
PPO (Hình 7)	1 lần huấn luyện, đánh giá 20 episode	Chưa có khoảng tin cậy giữa các seed huấn luyện



Hình 4. Mô hình chạy test: hiệu ứng thật so với các cách ước lượng trong ba chế độ spillover (chưa chạy trên simulator thật). Trái: 1 lần chạy, thanh là sai số chuẩn; giữa: 20 lần Monte Carlo; phải: 4 môi trường, thanh là độ lệch chuẩn.

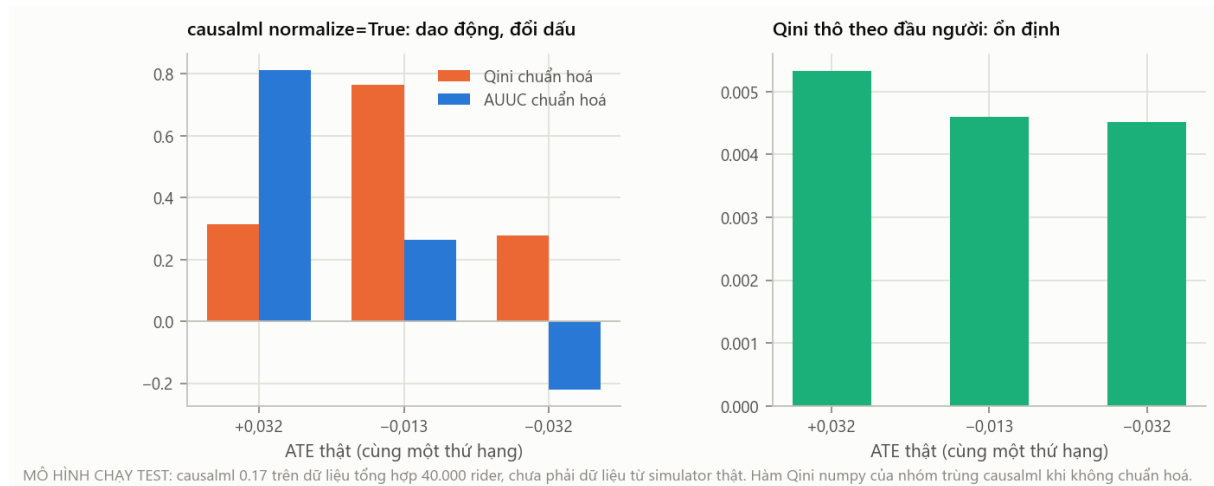
- **Chỉ trong cell** (1 lần chạy): chia ngẫu nhiên theo rider lệch khoảng **5 lần**; switchback hex $\times$ 2 giờ lệch ít hơn nhiều nhưng vẫn thấp hơn hiệu ứng thật khoảng 30%. Cần lặp nhiều seed trước khi kết luận.
- **Tài xế sang vùng kề** (CausalMARL, 20 lần Monte Carlo): ước lượng ngẫu thơ **sai cả dấu** ($+37,4 \pm 0,45$ so với $-2,67$).
- **Có carryover** (xp_gym, 4 môi trường): switchback ngẫu thơ hụt khoảng 70%; bộ ước lượng DN (Peng et al., 2025) gần đúng nhất về trung bình nhưng phân tán rất lớn ($\pm 0,710$).

► **Dùng cho simulator:** cần gán nhánh theo cụm (A/B, switchback) và chạy phản thực để có hiệu ứng thật; marketplace_sim.py đã có cả hai, bản rút gọn chỉ cần M10 và M13 ở dạng tối thiểu.



Hình 5. Mô hình chạy test: simulator của Farias et al. (2022) (nhóm đã vá, tham số chưa hiệu chỉnh, 2 seed mỗi cấu hình, chưa có khoảng tin cậy) trên dữ liệu taxi Manhattan: tác động thật của voucher 20% (vạch cam) và ước lượng của từng thiết kế thí nghiệm (chấm xanh).

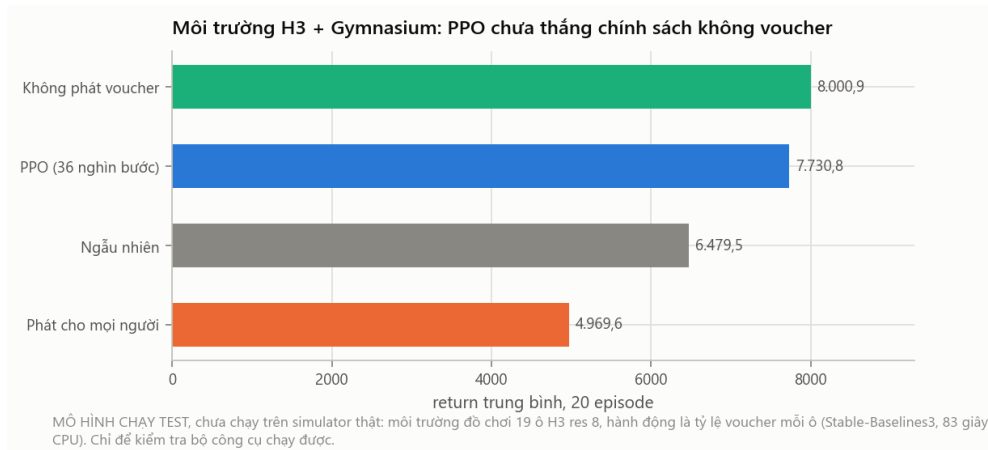
- Khi cung căng (70 xe), A/B theo rider báo +3,7 đến +4,1 điểm % (2 seed) trong khi tác động thật âm; switchback 15 phút gần sự thật nhất.
- Khi cung dư (250 xe), A/B phóng đại khoảng 1,5 lần.
- Với 2 seed, các chênh lệch này chỉ là định hướng; cần lặp ít nhất 20 seed và báo khoảng tin cậy trước khi dùng làm số liệu.



Hình 6. Mô hình chạy test (1 bộ dữ liệu tổng hợp): độ ổn định của thước đo Qini/AUUC khi ATE thay đổi, cùng một thứ hạng.

- Chế độ chuẩn hoá mặc định của causalml phóng to khi ATE gần 0 và làm AUUC **đổi dấu khi ATE âm**.
- Qini thô theo đầu người ổn định; hàm Qini của nhóm trùng causalml ở chế độ này. Khi điểm uplift có nhiều giá trị trùng, cần gộp theo giá trị (như scikit-learn) vì thứ tự dòng làm Qini dao động khoảng 14%.

► **Dùng cho simulator:** phần đánh giá ở tuần 2: báo cáo Qini thô, gộp giá trị trùng; $V(\pi)$ là tiêu chí chính. Hai phát hiện này được đưa vào concept note Qini/AUUC.



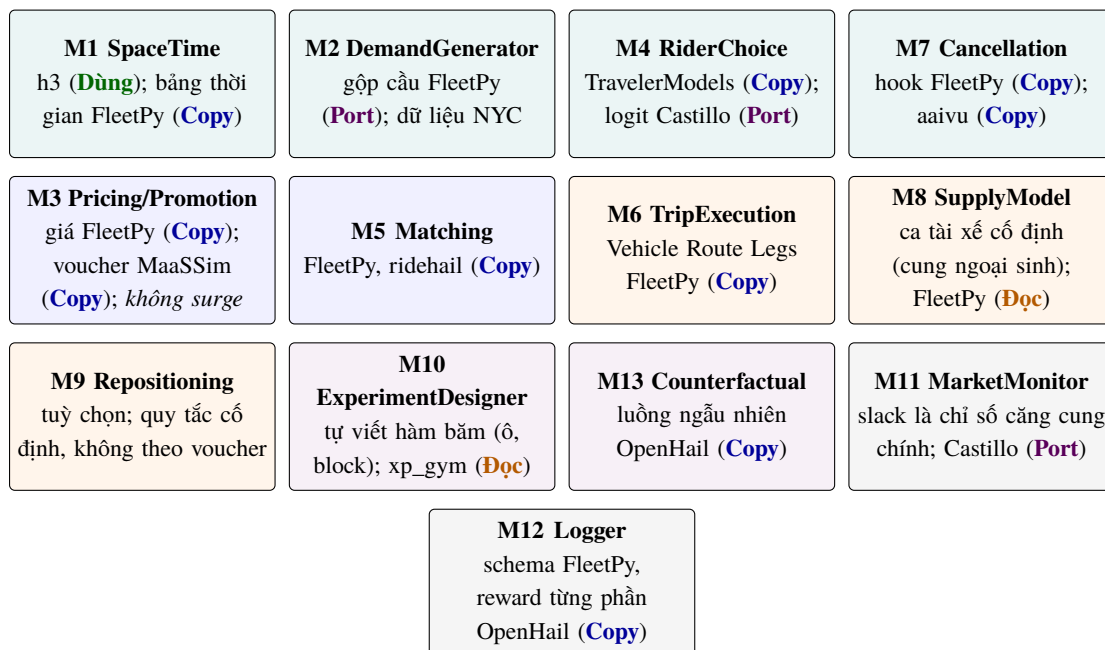
Hình 7. Mô hình chạy test (môi trường đồ chơi H3 + Gymnasium, 1 lần huấn luyện, đánh giá 20 episode): return của PPO so với các chính sách viết tay. Ngoài phạm vi đề bài, chỉ ghi lại để tham khảo.

- Bộ h3, Gymnasium và Stable-Baselines3 cài chung được và chạy đúng giao diện chuẩn.
- PPO chưa thắng chính sách “không phát voucher”. Học tăng cường và quyết định tuần tự nằm **ngoài phạm vi đề bài** (§6), nên nhóm **không đầu tư thêm** vào hướng này trong 5 tuần.

5 Áp dụng mã nguồn cho bản ABM rút gọn

Mục này chỉ áp dụng cho bản ABM rút gọn dùng để kiểm tra ngoài; simulator chính `marketplace_sim.py` không phụ thuộc vào các nguồn mã dưới đây. Các thành phần ngoài phạm vi (Bảng 1) đã được loại hoặc thay bằng quy tắc cố định.

5.1 Nguồn mã cho từng module



Hình 8. Nguồn mã cho 13 module của bản ABM rút gọn (màu theo khối chức năng: xanh lục – thể giới và khách hàng; xanh dương – chính sách nền tảng; cam – vận hành đội xe; tím – thí nghiệm; xám – giám sát và dữ liệu).

Bảng 7. Nguồn mã, giấy phép và cách dùng cho từng module.

Module	Nguồn và nội dung lấy	Giấy phép	Mức
M1 SpaceTime	h3-py: latlng_to_cell, grid_disk; FleetPy: bảng thời gian tính trước và cache OD	Apache; MIT	Dùng, Copy
M2 DemandGenerator	FleetPy: gộp cầu theo zone $\times$ 15 phút; fleet-sim: bộ lọc ngoại lai	MIT	Port
M3 Pricing/Promotion	FleetPy _compute_fare (chỉ phần theo thời gian, quãng đường); voucher $(1 - d) \cdot \text{giá}$ (MaaSSim). Không có surge (ngoài phạm vi, §6)	MIT	Copy
M4 RiderChoice	FleetPy TravelerModels; logit giá-ETA có hệ số giá luôn âm (Castillo, 2025)	MIT; CC-BY	Copy, Port
M5 Matching	FleetPy insertion; ridehail: ghép xe rảnh gần nhất	MIT	Copy
M6 TripExecution	FleetPy Vehicle Route Legs: giữ xe từ lúc ghép tới lúc trả	MIT	Copy
M7 Cancellation	FleetPy cancels_booking; aaviv: huỷ sau số lần chờ quá hạn	MIT; Apache	Copy
M8 SupplyModel	Số xe theo ca cho trước, hiệu chỉnh theo NYC; cung ngoại sinh với voucher. Utility tài xế (ridehail, MaaSSim) để sau vì thuộc incentive phía tài xế	MIT	Độc
M9 Repositioning	Tuỳ chọn. Nếu giữ: quy tắc cố định (vd. xe rảnh quay về ô gốc của ca), không phụ thuộc cầu hay voucher. Không dùng repositioning theo cầu (FleetPy, CausalMARL) vì làm cung nội sinh với voucher	—	—
M10 Experiment-Designer	Tự viết hàm bấm (ô, block) thành nhánh theo cơ chế của xp_gym; cluster-experiments chỉ đọc để đối chiếu	MIT; không có	Độc
M11 MarketMonitor	Chỉ số slack = $(\text{xe} - \text{đang chờ}) / \text{đang đón}$, nhãn wild goose chase (Castillo, 2025). Dùng làm chỉ số căng cung chính thay $\rho = I / \max(W, 1)$ của báo cáo thiết kế (không bảo hoà, có cơ sở từ Castillo)	CC-BY	Port
M12 Logger	Schema log FleetPy; OpenHail: trả từng thành phần reward	MIT	Copy
M13 Counterfactual	OpenHail: luồng số ngẫu nhiên riêng cho yêu cầu và xe (common random numbers)	MIT	Copy
Đánh giá	causalml (uplift, Qini), scikit-uplift (kiểm chéo)	Apache; MIT	Dùng

5.2 Thư viện dùng trực tiếp

Bảng 8. Bộ thư viện tối thiểu cho 5 tuần.

Thư viện	Giấy phép	Vai trò
numpy, pandas	BSD	Xử lý dữ liệu, simulator
LightGBM, scikit-learn	MIT; BSD	Mô hình nền cho meta-learner (đề bài: không cần deep learning)
causalml 0.17	Apache-2.0	Ước lượng uplift, Qini/AUUC
scikit-uplift 0.5	MIT	Kiểm chéo Qini, xử lý giá trị trùng
h3 4.5	Apache-2.0	Lưới hex, ô kê (M1); chỉ khi làm bản ABM rút gọn

Chưa cần trong 5 tuần: cluster-experiments, estimagic, Optuna, Gymnasium, Stable-Baselines3.

Với các kho **không có giấy phép** (RL_pricing, CausalMARL, xp_gym, InfSim, SwitchMDP): chỉ đọc cơ chế, ghi lại thành công thức kèm trích dẫn bài báo, rồi tự viết lại; không sao chép mã vào dự án.

6 Kết luận

Simulator chính của đề tài là `marketplace_sim.py`: nó đã có confounding, interference và hàm $V(\pi)$ chạy nhanh, đủ để trả lời RQ1–RQ3 trong 5 tuần. Không có mã mở nào thay được nó. Thiết kế 13 module chỉ được làm, nếu còn thời gian, dưới dạng **bản ABM rút gọn để kiểm tra ngoài**. Cho bản rút gọn đó:

- **FleetPy** (Engelhardt et al., 2022) là ứng viên tham chiếu kiến trúc (Python, MIT, còn bảo trì, tách engine khỏi chính sách), nhưng chỉ được cam kết sau một spike ≤ 1 ngày chạy ví dụ Manhattan;
- logit giá–ETA và chỉ số slack/wild goose chase lấy từ Castillo (2025); slack là chỉ số căng cung chính (M11);
- voucher theo MaaSSim; gán nhánh switchback tự viết theo cơ chế xp_gym; chạy phản thực theo cách của OpenHail;
- đánh giá bằng causalml (kiểm chéo scikit-uptlift), với $V(\pi)$ là tiêu chí chính.

Surge, repositioning theo cầu, incentive tài xế và học tăng cường nằm ngoài phạm vi đề bài (§6) nên được loại khỏi kế hoạch. Các lần chạy thử trên mô hình test (1 đến 20 lần chạy, chưa chạy trên simulator thật) gợi ý rằng chọn sai đơn vị ngẫu nhiên hoá có thể làm ước lượng lệch nhiều lần hoặc đổi dấu, phù hợp với Farias et al. (2022) và Peng et al. (2025). Các kết quả này cần được lặp lại với nhiều seed, kèm khoảng tin cậy, trên `marketplace_sim.py`.

Tài liệu tham khảo

- Juan Camilo Castillo. Who benefits from surge pricing? *Econometrica*, 2025. Gói tái lập: Zenodo 15191154.
- W. Chen, H. Shi, and J. Ke. HRSim: An agent-based simulation platform for high-capacity ride-sharing services. *arXiv preprint arXiv:2505.17758*, 2025.
- Roman Engelhardt, Florian Dandl, Arslan-Ali Syed, Yunfei Zhang, Fabian Fehn, Fynn Wolf, and Klaus Bogenberger. FleetPy: A modular open-source simulation tool for mobility on-demand services. *arXiv preprint arXiv:2207.14246*, 2022.
- Vivek Farias, Andrew A. Li, Tianyi Peng, and Andrew Zheng. Markovian interference in experiments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2206.02371.
- Nhóm đề tài (Simulator). Simulator gọi xe cho đề tài Supply-Aware Promotion Uplift: kiến trúc, tác tử và các module. Tài liệu nội bộ, 2026.
- Tianyi Peng, Naimeng Ye, and Andrew Zheng. Differences-in-neighbors for network interference in experiments. In *Proceedings of the ACM Conference on Economics and Computation (EC '25)*, 2025. arXiv:2503.02271.
- Chengchun Shi và cộng sự. A multi-agent reinforcement learning framework for off-policy evaluation in two-sided markets. *Annals of Applied Statistics*, 2023. Mã: <https://github.com/RunzheStat/CausalMARL>.